Habit Tracker Application

Phase 2 - Development & Reflection

Student: **Faruq Ishola Raji** / MatrNo: **4253139**

Object Oriented and Functional Programming with Python (DLBDSOOFPP01)

GitHub: https://github.com/rajifaruq2021-hue/oofpp_habits_project

Slide 2: Project Overview

What did I build?

I built a command-line Habit Tracking application in Python that allows users to create, manage, and analyse their daily and weekly habits. The application is designed around two core Python style of programmings:

* **Object-Oriented Programming (OOP)** - the Habit class models real-world habit data and behaviour
* **Functional Programming** - the analytics module uses filter(), map(), and max() to analyse habits

Feature	Details
Create habits	Users define habits with a name, description, and period
Check off	Mark a habit as done - timestamp recorded automatically
Streak tracking	Current and longest streaks calculated per habit
Analytics	4 required analytics functions + 2 bonus functions
Saving data	SQLite database - data survives between sessions
Unit tests	20 tests using pytest - all passing
Pre-defined data	5 habits with 4 weeks of example check-off history

Slide 3: Application Architecture

How the modules connect

The application is split into 5 focused modules. Each module has one clear job:

Module	Role	Key Contents
main.py	Entry point / CLI Menu	Interactive numbered menu loop
habit.py	OOP - Habit Class	Habit data + streak methods
analytics.py	Functional Analytics	filter(), map(), max() functions
db.py	Database Layer	SQLite save / load / delete
seed_data.py	Test Fixtures	5 habits + 4 weeks of data
tests/	Unit Test Suite	20 tests (pytest / unittest)

Data flows in one direction: the user interacts with **main.py** which uses **habit.py** objects which then goes to persisted by **db.py** into **habits.db**. The analytics module reads (never modifies) the habit objects.

Slide 4: Object-Oriented Design

The Habit Class (habit.py)

I encoded the concept of a habit as a Python class. This is the most natural fit because a habit is a real-world entity with its own data and behaviour:

* **Attributes:** name, description, periodicity, creation_date, checkoffs
* **checkoff()** - records a completion event with the current timestamp
* **get_completed_periods()** - converts timestamps into unique day/week labels
* **get_current_streak()** - counts consecutive periods from the most recent backwards
* **get_longest_streak()** - scans all history to find the all-time best streak
* **to_dict() / from_dict()** - converts the habit for database storage

Example - creating a daily habit in code:

```
from habit import Habit my_habit = Habit(name='Drink Water', description='2 litres per day', periodicity='daily') my_habit.checkoff() # marks it done right now print(my_habit.get_current_streak()) # prints: 1
```

Slide 5: Functional Programming

The Analytics Module (analytics.py)

The analytics module is written in the **functional programming** style. Every function is pure - it takes a list of habits in, computes something, and returns a result. No data is ever modified.

Assignment Requirement	My Function	FP Tool Used
Return all currently tracked habits	get_all_habits()	-
Return habits with same periodicity	get_habits_by_periodicity()	filter()
Longest streak of ALL habits	get_longest_streak_all()	map() + max()
Longest streak for a GIVEN habit	get_longest_streak_for_habit()	next() + generator

Example - using filter() to get only daily habits:

```
daily = list(filter( lambda h: h.periodicity == 'daily', all_habits ))
```

Slide 6: Data Saving data

SQLite Database (db.py)

I chose **SQLite** via Python's built-in *sqlite3* module. It requires no installation - it is part of Python. All habit data is stored in a single file: **habits.db**.

The database has 2 tables:

```
Table: habits id | name | description | periodicity | creation_date Table:
checkoffs id | habit_name (FK links to habits.name) | timestamp
```

* **save_habit()** - saves or updates a habit and all its check-offs
* **load_all_habits()** - loads all habits back into Habit objects on startup
* **delete_habit()** - removes a habit and its check-offs permanently
* **habit_exists()** - checks for duplicates before inserting

Slide 7: Test Fixtures

5 Pre-defined Habits (seed_data.py)

To meet the requirement of providing example data for testing, I pre-loaded 5 habits with 4 weeks of realistic check-off history (January 1-28, 2024):

#	Habit Name	Type	4-Week Pattern
1	Drink Water	Daily	27/28 days - missed Jan 17
2	Exercise	Daily	25/28 days - missed 3 days
3	Read a Book	Daily	27/28 days - missed Jan 18
4	Weekly Review	Weekly	4/4 weeks - perfect streak
5	Clean the House	Weekly	3/4 weeks - missed week 3

I made the data intentionally imperfect (a few missed days) so it is realistic and useful for testing streak break logic.

Slide 8: Unit Tests

20 Tests - All Passing (tests/)

I wrote 20 automated unit tests using Python's *unittest* framework, runnable with *pytest*. The tests verify the two most critical components:

Test File	Tests	What is tested
tests/test_habit.py	10	Habit creation, checkoff, daily/weekly streaks, longest streak, to_dict/from_dict roundtrip
tests/test_analytics.py	10	All analytics functions: filter by periodicity, longest streak, habit lookup, summary

Run the full test suite with:

```
python -m pytest tests/ -v Result: ======================== 20 passed in 0.09s ========================
```

Slide 9: How to Install and Run

Step-by-step guide

Requirements: Python 3.7+ and pytest (only external dependency)

Step 1 - Clone from GitHub or unzip the project folder

```
git clone https://github.com/rajifaruq2021-hue/oofpp_habits_project
```

Step 2 - Navigate into the project folder

```
cd oofpp_habits_project
```

Step 3 - Install the only dependency

```
pip install pytest
```

Step 4 - Run the application

```
python main.py
```

Step 5 - Run the unit tests

```
python -m pytest tests/ -v
```

On first launch the app creates *habits.db* automatically and loads the 5 pre-defined habits. The interactive menu guides the user through all available actions using simple numbered options.

Slide 10: Conclusion & Reflection

What I built and what I learned

I successfully built a fully functional Habit Tracker application that meets all requirements defined in the assignment brief. Reflecting on the process:

* **OOP** - Modelling a habit as a class made the code clean and reusable. Each Habit object knows its own data and how to calculate its own streak.

* **Functional Programming** - Using filter(), map(), and max() in the analytics module made the analysis code short and clear, readable, and does not change any data.

* **SQLite** - Choosing a real database over JSON files added complexity but taught important concepts: tables, foreign keys, and SQL.

* **Unit Testing** - Writing 20 tests before and after coding gave me confidence that the streak logic works correctly in all unusual situations.

GitHub Repository: https://github.com/rajifaruq2021-hue/oofpp_habits_project

Student: Faruq Ishola Raji / MatrNo: 4253139